

Cấu trúc dự án trên IntelliJ IDEA (Độc lập, Không đụng chạm code)

Để 3 người làm song song mà không bị xung đột code khi ghép (merge conflict), dự án chia làm **3 module** riêng biệt hoặc 1 project với 3 package tách bạch ranh giới:

Plaintext

ebanking-rmi/

```
└─ src/
|
| └─ common/      <-- THỐNG NHẤT TRƯỚC (Không ai được tự ý sửa sau Day 1)
|   |
|   | └─ models/   (Account, Bill, Transaction, Saving - implements Serializable)
|   |   └─ rmi/     (IBankService.java, IClientCallback.java)
|   |
|   |
|   └─ server/     <-- NGƯỜI 1 & NGƯỜI 2 LÀM TRÊN CÁC SERVICE KHÁC NHAU
|       |
|       | └─ db/    (DatabaseConnection.java)
|       |
|       | └─ dao/    (AccountDAO, BillDAO, TransactionDAO, SavingDAO)
|       |
|       | └─ service/ (CoreBankServiceImpl, ExtraBankServiceImpl)
|       |   └─ ServerMain.java (Khởi tạo Registry 1099, bind Service)
|       |
|       |
|       └─ client/  <-- MỖI NGƯỜI TỰ CODE MÀN HÌNH SWING RIÊNG CỦA MÌNH
|           |
|           | └─ callback/ (ClientCallbackImpl.java)
|           |
|           | └─ view/
|           |   |
|           |   | └─ auth/ (LoginForm.java, RegisterForm.java - Dev 1)
|           |   |
|           |   | └─ transfer/ (TransferForm.java - Dev 1)
|           |   |
|           |   | └─ bill/ (BillPayForm.java - Dev 2)
|           |   |
|           |   | └─ saving/ (SavingForm.java - Dev 3)
```

```
| | └─ admin/      (AdminDashboardForm.java - Dev 3)
| | └─ ClientMain.java
```

Phân chia công việc chi tiết cho 3 người

Người 1: Xương sống Hệ thống, Tài khoản & Chuyển tiền Real-time (Core Architecture & Callback)

- **Góc nhìn đời sống:** "Người xây nền móng ngân hàng". Đảm nhận việc mở tài khoản, đăng nhập, bảo vệ kết tiền và tính năng chuyển tiền "ting ting" lập tức sang máy người khác giống hệt app ngân hàng thật.
- **Thuật ngữ Lập trình mạng & Kỹ thuật:**
 - Khởi tạo RMI Registry (cổng 1099) và cấu hình Remote Service.
 - Quản lý phiên kết nối tập trung (Session Management) bằng `ConcurrentHashMap<String, IClientCallback>`.
 - Cài đặt **RMI Callback Pattern**: Server chủ động gọi ngược hàm `notifyBalanceChange()` trên máy nhận tiền (Event-driven Notification) thay vì để Client dùng cơ chế Polling (kéo/hỏi liên tục gây nghẽn mạng).
 - Kiểm soát tranh chấp tài nguyên (Concurrency Control) và Race Condition bằng `synchronized` kết hợp Database Transaction (`setAutoCommit(false)`, `commit()`, `rollback()`).
- **Phạm vi code cụ thể:**
 - `common/`: Định nghĩa `IBankService` (các hàm login, logout, transfer, getBalance) và `IClientCallback`.
 - `server/`: `DatabaseConnection.java`, `AccountDAO.java`, xử lý luồng `transfer()`.
 - `client/`: `LoginForm.java`, `TransferForm.java`, `ClientCallbackImpl.java`.
- **Lỗi hổng cốt tử cần tự kiểm thử (Test Cases):**
 1. *Click tặc (Double-click)*: Dùng 2 thread trên client gửi cùng lúc 2 lệnh chuyển tiền với số dư chỉ đủ cho 1 lệnh → Tiền không bao giờ được phép âm.
 2. *Dead Callback Reference*: Client B đang online rồi tắt ngang app (hoặc rút dây mạng) → Client A chuyển tiền cho B → Server phải bắt `RemoteException`, tự động gỡ B khỏi danh sách online, không để luồng Server bị đơ (hang thread).

Người 2: Thanh toán Hóa đơn & Lịch sử Giao dịch (3-Tier Transaction & Serialization)

- **Góc nhìn đời sống:** "Nhân viên quầy thu ngân & in sao kê". Nhận mã tiền điện/nước/internet để gạch nợ, đồng thời ghi lại từng đồng tiền ra vào để khách mở app là xem được toàn bộ lịch sử chi tiêu.
- **Thuật ngữ Lập trình mạng & Kỹ thuật:**
 - Triển khai kiến trúc **3 lớp (3-tier Architecture)** chuẩn: Client không chạm trực tiếp vào SQL; toàn bộ truy vấn qua Server RMI để chống giả mạo số tiền hay dữ liệu hóa đơn.
 - Quản lý giao dịch nguyên tử (**ACID Transaction**): Khi trả hóa đơn, lệnh trừ tiền ví khách và lệnh đổi trạng thái hóa đơn (UNPAID → PAID) phải nằm chung một Transaction.
 - Tuần tự hóa đối tượng (**Object Serialization**): Chuyển danh sách List<Transaction> phức tạp thành luồng byte truyền qua mạng TCP/IP về Client hiển thị trên JTable.
- **Phạm vi code cụ thể:**
 - server/: BillDAO.java, TransactionDAO.java, các hàm queryBill(billCode), payBill(billCode, accountNum), getTransactionHistory(accountNum).
 - client/: BillPayForm.java (Tra cứu mã, hiển thị thông tin hóa đơn, bấm thanh toán), TransactionHistoryForm.java (bảng lịch sử, bộ lọc ngày tháng, nút xuất sao kê ra file text/CSV).
- **Lỗi hỏng cốt tử cần tự kiểm thử (Test Cases):**
 1. *Thanh toán lưỡng chừng (Dirty State):* Thử ngắt kết nối database ngay sau khi trừ tiền tài khoản khách → Lệnh phải rollback, không được để xảy ra tình trạng "tiền mất mà hóa đơn vẫn còn nợ".
 2. *Thanh toán 2 lần trên 1 hóa đơn:* Mở 2 máy khách cùng tra cứu mã EVN_HANOI_01 và bấm trả cùng lúc → Chỉ 1 máy thành công, máy kia nhận thông báo "Hóa đơn đã được thanh toán trước đó".
 3. *Lỗi Serializable:* Quên implements Serializable ở model Bill hoặc Transaction dẫn đến văng lỗi NotSerializableException trên đường truyền mạng.

Người 3: Sở tiết kiệm tự sinh lãi & Bảng điều khiển Admin (Multi-threading & Remote Control)

- **Góc nhìn đời sống:** "Thủ quỹ & Giám đốc kỹ thuật". Tự động tính lãi gửi tiết kiệm theo thời gian định kỳ cho khách, đồng thời cung cấp màn hình riêng cho Sếp xem có bao nhiêu người đang dùng mạng và khóa các tài khoản đáng ngờ.
- **Thuật ngữ Lập trình mạng & Kỹ thuật:**
 - Đa tiến trình ngầm (**Multi-threading / Scheduled Task**): Chạy ScheduledExecutorService hoặc Thread nền trên Server để đếm chu kỳ (giả lập 30 giây hoặc 1 phút = 1 kỳ hạn) nhằm tính lãi tích lũy và kích hoạt Callback báo về Client khi phát sinh lãi.
 - Giám sát kết nối mạng từ xa (**Network Session Monitoring**): Lấy trực tiếp danh sách kết nối active từ onlineClients của Người 1 để render lên giao diện Admin.
 - Lệnh điều khiển cưỡng chế từ xa (**Remote Revocation**): Admin bấm "Khóa tài khoản" hoặc "Kick" → Server triệu gọi hàm callback bắt buộc Client của nạn nhân tự hủy phiên và thoát về màn hình Login.
- **Phạm vi code cụ thể:**
 - server/: SavingDAO.java, InterestCalculatorTask.java (tiến trình nền tính lãi), các hàm RMI: openSavingAccount(), settleSaving(), getOnlineUsers(), lockUser().
 - client/: SavingForm.java (Mở sổ, theo dõi tiền lãi nhảy trực tiếp), AdminDashboardForm.java (Xem danh sách online, tổng tiền hệ thống, nút đóng băng/đá văng tài khoản).
- **Lỗi hỏng cốt tử cần tự kiểm thử (Test Cases):**
 1. *Lỗi rò rỉ luồng (Thread Leak)*: Khách đã tắt toán sổ tiết kiệm nhưng Thread tính lãi của sổ đó trên Server vẫn âm thầm chạy ngầm, gây tốn CPU và lỗi cộng dồn tiền sai.
 2. *Xung đột khóa (Deadlock) giữa luồng tính lãi và luồng rút tiền*: Luồng tính lãi đang ghi sổ dư thì khách bấm tắt toán sổ → Cần dùng khóa đồng bộ để không phát sinh sai lệch sổ dư cuối cùng.

Ma trận Trách nhiệm & Hướng dẫn Tích hợp (Merge Code)

Thành viên	Phân hệ nghiệp vụ	Thành phần Server / DB	Giao diện Client (Swing)	Trọng tâm Network / Hệ thống
Người 1	Auth + Chuyển khoản	accounts, Registry, Session Pool	LoginForm, TransferForm	RMI Callback, Concurrency, Connection Handling
Người 2	Hóa đơn + Sao kê	bills, transactions, ACID Logic	BillPayForm, HistoryForm	Object Serialization, JDBC Transaction Rollback
Người 3	Tiết kiệm + Admin	savings, Background Worker	SavingForm, AdminForm	Multi-threading, Remote Session Revocation

Quy tắc ghép code vào cuối tuần:

- Ngày 1 (Chốt Interface):** Cả nhóm ngồi lại 30 phút để chốt toàn bộ các hàm trong common/rmi/IBankService.java và IClientCallback.java. Khi đã push lên Git, **không ai được đổi tên hàm hay kiểu dữ liệu trả về.**
- Trong quá trình làm:** Người nào viết hàm nào trên Service thì tự viết DAO và giao diện Swing tương ứng của hàm đó.
- Ngày cuối (Ghép code):**
 - Người 1 push khung ServerMain và kết nối Registry.
 - Người 2 và Người 3 chỉ cần copy các khối phương thức (methods) đã test chạy độc lập của mình paste vào BankServiceImpl và gắn các panel giao diện vào menu chính (DashboardForm). Không có nguy cơ conflict code vì mỗi người sở hữu hoàn toàn các file View và DAO riêng biệt.

Bước 1: Thêm 2 bạn kia vào làm cộng tác viên (Collaborators)

Hai bạn kia cần có quyền ghi vào repo thì mới push nhánh lên được:

1. Lên web GitHub, mở repo vừa tạo.
2. Bấm vào **Settings** (tab bánh răng ở góc trên).
3. Ở cột bên trái, chọn **Collaborators** > Bấm nút xanh **Add people**.
4. Nhập username GitHub hoặc email của Bạn 2 và Bạn 3 để gửi lời mời.
5. **Nhắc 2 bạn:** Check email hoặc vào thẳng link repo bấm **Accept invitation** (nếu không chấp nhận thì 2 bạn đó không push code lên được).

Bước 2: Tạo nhánh riêng của bạn (Người 1) để bắt đầu code

Nhánh main giờ là nhánh chuẩn, **bạn không được code thẳng trên main nữa**. Bạn mở Terminal ở máy bạn và tạo nhánh riêng:

Bash

Tạo và chuyển sang nhánh làm việc của Người 1

```
git checkout -b feature/core-auth-transfer
```

Từ giờ, bạn chỉ việc code các phần của bạn (LoginForm, TransferForm, RMI Registry, Callback, ...). Hàng ngày code xong thì cứ commit và push lên nhánh này:

Bash

```
git add .
```

```
git commit -m "feat: setup RMI Registry va xu ly dang nhap"
```

```
git push origin feature/core-auth-transfer
```

Bước 3: Gửi tin nhắn hướng dẫn cho Bạn 2 và Bạn 3

Bạn copy toàn bộ đoạn tin nhắn dưới đây gửi vào nhóm chat Zalo/Messenger của nhóm để 2 bạn bắt đầu:

HƯỚNG DẪN BẮT ĐẦU DỰ ÁN EBANKING:

1. Mọi người check email chấp nhận lời mời vào repo GitHub này trước: <DÁN BẠN CỦA LINK REPO VÀO ĐÂY>.
2. Mở Terminal trên máy các bạn, chạy các lệnh sau để lấy code về:

Bash

```
git clone <DÁN LINK REPO CỦA BẠN VÀO ĐÂY>
```

```
cd <TEN_THU_MUC_DU_AN>
```

3. Tạo nhánh riêng của từng người (ai làm phần nào thì chạy đúng dòng đó):

- **Người 2 (Hóa đơn & Lịch sử GD):**

Bash

```
git checkout -b feature/bill-transaction
```

- **Người 3 (Tiết kiệm & Admin):**

Bash

```
git checkout -b feature/saving-admin
```

4. **Quy tắc làm việc hàng ngày:**

- Chỉ code trên các file được phân công, **tuyệt đối không sửa các file trong thư mục common/**.
- Cuối ngày đẩy code lên nhánh của mình:

Bash

```
git add .
```

```
git commit -m "mô tả việc vừa làm hôm nay"
```

```
git push origin <tên_nhánh_của_mình>
```

- **Tuyệt đối KHÔNG push thẳng lên main.** Khi nào xong tính năng thì lên GitHub tạo Pull Request để trưởng nhóm duyệt.

Như vậy là xong phần thiết lập! Bây giờ bạn chỉ việc quay lại IntelliJ IDEA và bắt đầu code phần của Người 1 trên nhánh feature/core-auth-transfer.